

10. Header Linked List

–Traverse In Order

Program:

```
void traverseInOrder()
{
    if (head->next == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }

    Node *temp = head->next; // Skip header
    cout << "Header -> ";

    while (temp != nullptr)
    {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}

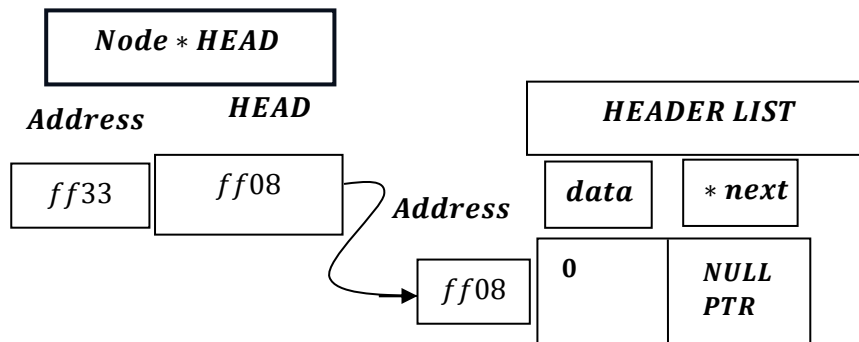
... .

case 8:
    traverseInOrder();
    break;
```

Program Analysis

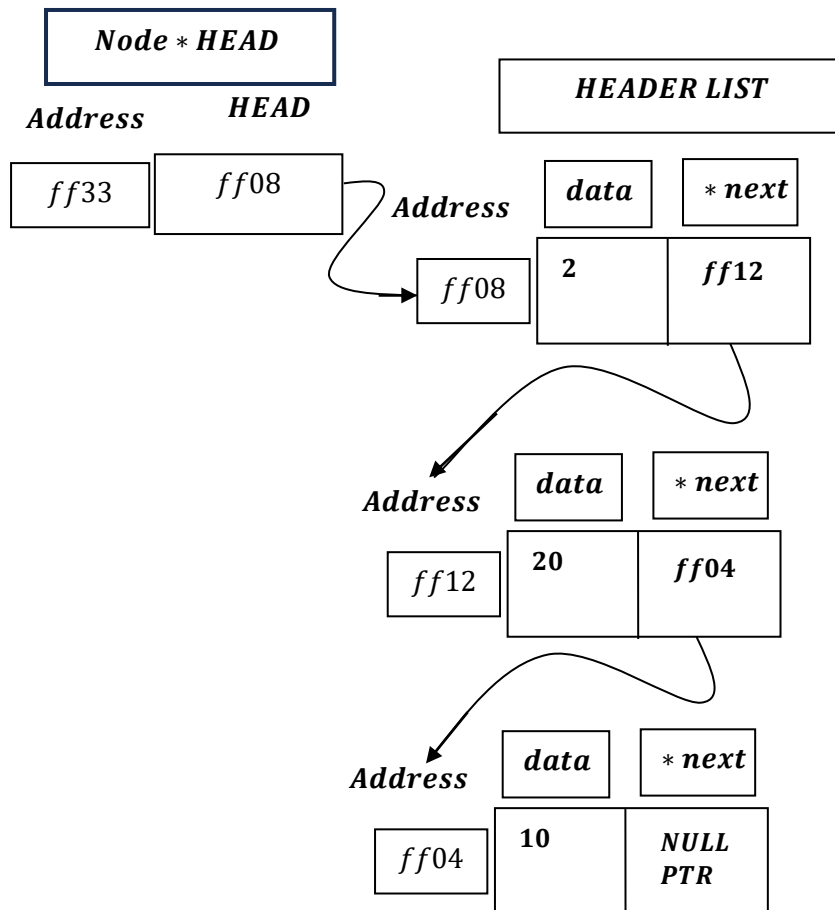
```
if (head->next == nullptr)
{
    cout << "List is empty. Nothing to delete." << endl;
    return;
}
```

i.e.



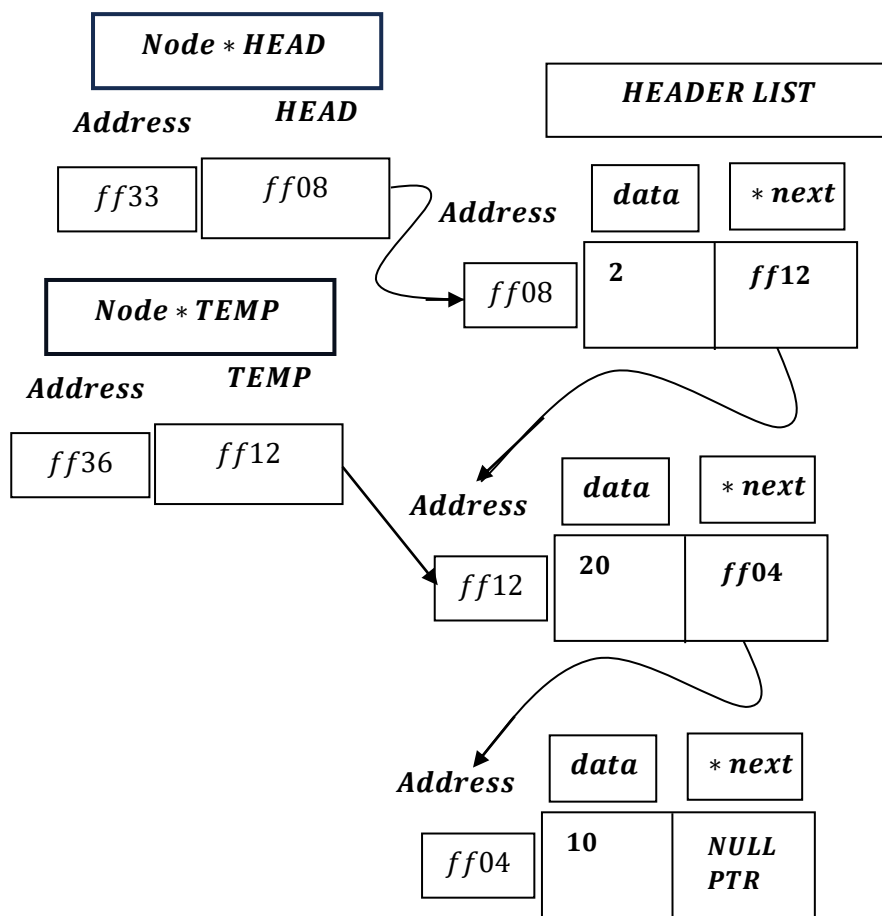
head → *next* = *NULLPTR* , *head* → *Data* = 0 , then list is empty .

If List is not Empty



```
Node *temp = head->next; // Skip header
```

Node * TEMP = HEAD → NEXT = ff12.



```
cout << "Header -> ";

while (temp != nullptr)
{
    cout << temp->data << " -> ";
    temp = temp->next;
}
```

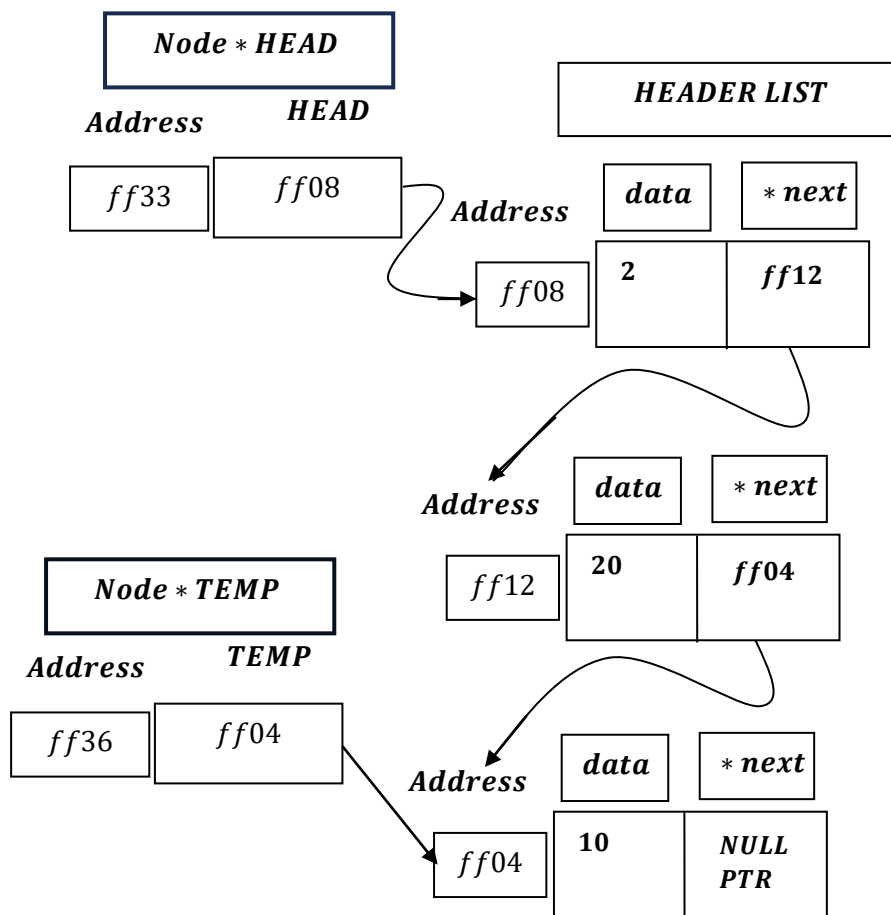
Output : ``Header →

Temp : ff12 ≠ NULLPTR :

Print : TEMP → DATA : 20 →

Temp = TEMP → NEXT = ff04.

i.e., OUTPUT: HEADER → 20 →

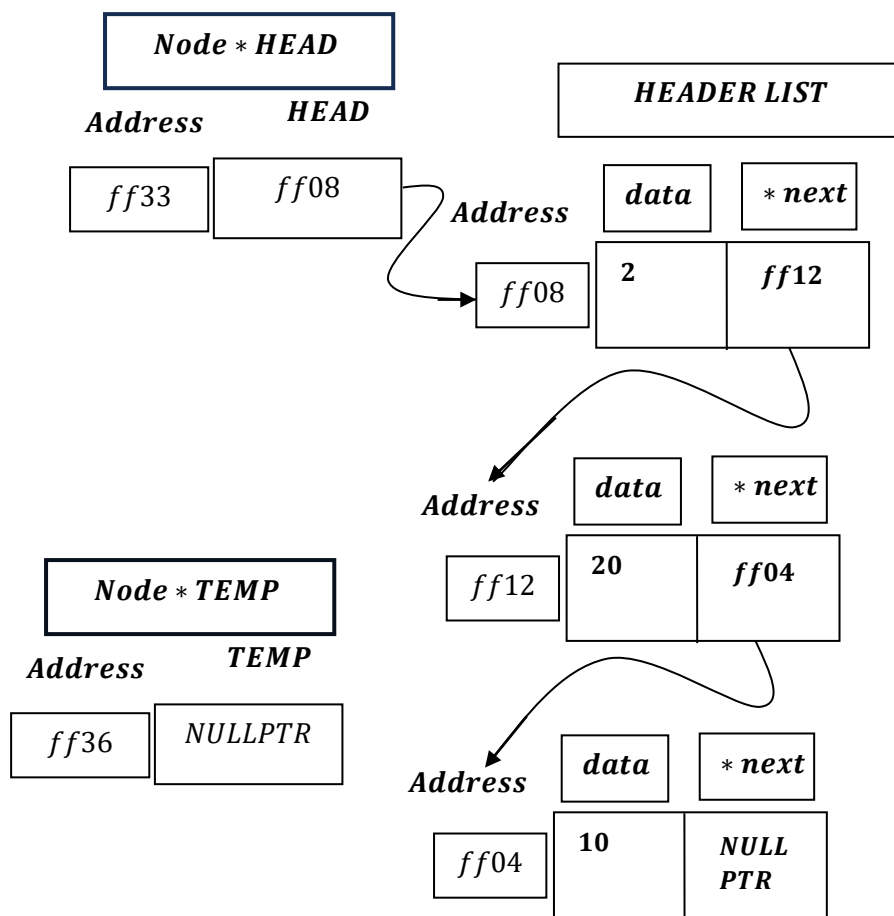


Temp : ff04 ≠ NULLPTR :

Print : TEMP → DATA : 10 →

Temp = TEMP → NEXT = NULLPTR.

i.e., OUTPUT: HEADER → 20 → 10 →



```
cout << "NULL" << endl;
```

i.e., OUTPUT: HEADER → 20 → 10 → NULL.
